

Centralized Report Management & Automated Dispatch Platform

Project Status Report — Database & Backend Progress
Prepared for the development team | Stack: Python (FastAPI) · Microsoft SQL Server · Oracle · DB2 · Windows Task Scheduler
Version 1.0 · July 2026

1. Project Overview

The platform is a centralized reporting hub used across 26 applications (both data sources and consumers). It stores report definitions, connects to multiple database engines (Microsoft SQL Server, Oracle, DB2), executes predefined queries, generates report files (Excel, PDF, CSV), and automatically dispatches them by email on a schedule. The design is configuration-driven: report name, SQL query, target database, schedule, output format, and recipients are all stored as data, so adding or changing a report requires no code change.

This report summarizes everything completed so far in the **database** and the **backend code**, the current test status, and the next phase broken into modules.

2. Database — Completed

The SQL Server database **ReportManagementDB** has been created and deployed via three scripts (schema, queries, and stored procedures/functions). It is fully operational.

2.1 Schema — 10 Tables

Table	Purpose
Users	Application accounts, roles (admin/viewer), hashed passwords, status.
Applications	Registry of the 26 systems (source / consumer / both).
Connections	Target databases: connection_type (sqlserver/oracle/db2) + encrypted credentials.
Reports	Central report definition: name, SQL query, connection, owner.
ReportFormats	Output formats per report (xlsx / pdf / csv).
User_Reports	Many-to-many: which users may access a report.
Schedules	Periodicity (daily/weekly/monthly/cron), next_run.
Recipients	Report-to-email mapping for automated dispatch.
ReportRuns	Execution audit: status, timing, row count, file path, errors.
Logs	User-action audit (login/create/edit/delete/download/execute).

2.2 Stored Procedures & Functions

All data access is funnelled through database objects — 26 stored procedures and 6 functions:

- **CRUD procedures** for every table: usp_User_*, usp_Application_*, usp_Connection_*, usp_Report_*, usp_Schedule_*, usp_Recipient_*, plus report-access grant/revoke.
- **Automation core:** usp_GetDueReports / ufn_GetDueReports, usp_ReportRun_Start, usp_ReportRun_Complete, usp_AdvanceSchedule.
- **Secure functions:** ufn_IsActionAllowed (RBAC), ufn_UserHasReportAccess, ufn_GetUserReports, ufn_GetReportFormats, ufn_GetReportRecipients — read-only, never expose secrets.
- **Credential handling:** usp_Connection_SetSecret / usp_Connection_GetSecret keep passwords encrypted; the secret is returned only on the dedicated internal path.

2.3 Security Model

- Two least-privilege roles (**app_role**, **automation_role**) are granted EXECUTE only and are **DENIED** direct INSERT/UPDATE/DELETE/SELECT on all base tables.
- Ownership chaining lets the procedures operate on the tables while the app itself cannot touch them directly — shrinking the SQL-injection surface and keeping all data logic in one audited place.
- Passwords are hashed (bcrypt); database credentials are stored encrypted (VARBINARY) and decrypted only at runtime.
- Every table uses CHECK constraints and foreign keys to enforce valid data.

3. Backend Code — Completed

The backend is being built in FastAPI, phase by phase, in a strictly modular way. A key architectural rule is enforced throughout: **the database is accessed only through the stored procedures and functions above** — there is no ORM and no raw table SQL. Phases 0 and 1 are complete and fully tested.

3.1 Phase 0 — Project Scaffolding & Foundations

Module	Key functions / classes	Result
app/config.py	Settings, get_settings() — env-driven config singleton	Done
app/core/logging_config.py	configure_logging(), get_logger() — JSON-line logging	Done
app/core/errors.py	AppError hierarchy + FastAPI exception handlers (no leaks)	Done
app/main.py	FastAPI app, GET /health	Done

3.2 Phase 1 — Database Connectivity & Stored-Procedure Caller

Module	Key functions / classes	Result
app/db/mssql.py	get_connection(), _connect(), DatabaseConnectionError — pooled connection with bounded retry on transient errors	Done
app/db/proc.py	call_proc(), call_proc_scalar(), query_function(), _build_exec_sql(), _rows_to_dicts(), _validate_proc_name(), _validate_param_name()	Done

- Every DB call now flows through call_proc / call_proc_scalar / query_function — parameterized values only, SQL text built solely from validated identifiers.
- Connections always close; only transient failures retry (a bug that retried non-transient failures was caught by the tests and fixed).
- Encrypted credential columns (VARBINARY) are passed through as raw bytes and never logged.

3.3 Test Status

Phase	Tests	Result
Phase 0	13 unit tests (config, logging, errors, health)	13 passed
Phase 1	17 tests (5 connection, 12 proc caller; 1 opt-in integration)	16 passed, 1 skipped
Combined suite	Full repo, no regressions	29 passed, 1 skipped

4. Current Status Snapshot

- **Database:** complete and deployed — schema, procedures, functions, and least-privilege security in place.
- **Backend:** Phases 0–1 complete — runnable FastAPI app, clean error handling, logging, and a safe, tested data-access layer that talks only to stored procedures.
- **Quality:** 29 tests passing, 1 integration test skipped (needs a live DB), zero known failures.
- **Next up:** Phase 2 — the Security Core (password hashing, credential encryption, JWT).

5. Next Phase — Phase 2: Security Core

Objective: Build the cryptographic building blocks that authentication (Phase 3) and the connection registry (Phase 5) depend on: secure password hashing, symmetric encryption for database credentials, and JWT token creation/verification. Nothing in this phase touches business logic — it produces small, heavily-tested primitives.

Why now: Authentication must sit on top of finished, verified crypto. Building these primitives first, in isolation, means the login and credential-encryption flows can be assembled safely afterwards.

Phase 2 is divided into three modules:

Module 2.1 — `app/core/security.py` (Password Hashing & JWT)

- **`hash_password(plain)`** -> **str** — bcrypt hash of a password.
- **`verify_password(plain, hashed)`** -> **bool** — constant-time verification.
- **`create_access_token(subject, role, expires_minutes)`** -> **str** — signed JWT embedding sub, role, and exp.
- **`decode_access_token(token)`** -> **dict** — verifies signature and expiry; raises `AuthError` on failure.

Notes: bcrypt truncates at 72 bytes (document or pre-hash long inputs); expiry driven by `JWT_EXPIRE_MINUTES`; invalid and expired tokens both raise the existing `AuthError`.

Module 2.2 — `app/core/crypto.py` (Credential Encryption)

- **`encrypt_secret(plain)`** -> **bytes** — symmetric (Fernet/AES) encryption of a DB password; the bytes are stored via `usp_Connection_SetSecret`.
- **`decrypt_secret(blob)`** -> **str** — decrypts the VARBINARY read back from `usp_Connection_GetSecret`; used only by the engine at connection time.
- **Startup validation** — verify `CRED_ENCRYPTION_KEY` is a well-formed 32-byte url-safe base64 key, so a bad key fails at startup rather than at first report run.

Module 2.3 — Tests (`tests/test_security.py`, `tests/test_crypto.py`)

- Hash verifies and is salted (two hashes of the same input differ); wrong password fails.
- JWT round-trips subject + role; an expired or tampered token raises `AuthError`.
- `encrypt_secret` → `decrypt_secret` round-trips; ciphertext is never equal to plaintext.
- Explicit check that no secret, key, or token is ever written to logs or error messages.

Definition of Done (Phase 2)

- All hashing, encryption, and JWT functions implemented with type hints and docstrings.
- Both test files pass; combined repo suite stays green with no regressions.
- `CRED_ENCRYPTION_KEY` validated at startup; no secret ever logged.
- A MODULE REPORT delivered after each of the three modules (function inventory, tests, pass/fail feedback).

6. Remaining Roadmap (after Phase 2)

Phase	Focus
Phase 3	Authentication & user management (login, JWT issuance, user CRUD).
Phase 4	Authorization / RBAC enforcement on every route.
Phase 5	Applications & Connections registry (encrypted credentials).
Phase 6	Report & configuration management (reports, formats, recipients, schedules).
Phase 7	Multi-database execution engine (SQL Server / Oracle / DB2).
Phase 8	Report generation (Excel / PDF / CSV).
Phase 9	Automation script, scheduling & PowerShell email dispatch.
Phase 10	Cross-cutting hardening, logging/audit, and end-to-end integration.